

• PRODUCTION BLUEPRINT • VERIFIED • LICENSE-CLEAN

Self-Hosted Tamil / English / Tanglish TTS

Four verified, production-grade architectures for self-hosted Text-to-Speech supporting Tamil, English, and code-mixed Tanglish at **≤500 ms p99 TTFA** and 15–20 concurrent streams. Fully self-hosted • Commercially deployable • No MAU / revenue gating.

VERSION

v1.1 • Aug 2026

SOLUTIONS

4 Architectures

LANGUAGES

ta • en • Tanglish

LATENCY TARGET

p99 ≤ 500 ms

Self-Hosted Tamil / English / Tanglish TTS — Production Architecture Blueprint

Four verified, license-clean, production-grade solutions for self-hosted Text-to-Speech supporting Tamil, English, and code-mixed Tanglish at $\leq 500\text{ms}$ p99 TTFA and 15–20 concurrent streams.

FIELD	DETAIL
Version	1.1
Date	August 25, 2026
Status	Verified against upstream license files
Languages	Tamil (ta), English (en), Tanglish (ta-en code-mix)
Constraint	Fully self-hosted · Commercially deployable · No MAU/revenue gating

Table of Contents

- 1. [Executive Summary](#)
- 2. [Requirements & Evaluation Criteria](#)
- 3. [Solution 1 — IndicF5 Modular Polyglot Pipeline \(Recommended Production Backbone\)](#)
- 4. [Solution 2 — Indic Parler-TTS Single-Backbone End-to-End](#)
- 5. [Solution 3 — Hybrid Tiered Router \(Cost-Optimized\)](#)
- 6. [Solution 4 — VITS + FastSpeech2 Cascaded Quick-Win Stack \(Fastest Path to Endpoint\)](#)
- 7. [Cross-Solution Comparison \(All Four\)](#)
- 8. [Decision Matrix — Which Solution to Choose](#)
- 9. [Implementation Roadmap — Phased Rollout](#)
- 10. [Risks, Exclusions & Mitigations](#)
- 11. [Appendices](#)

1. Executive Summary

This document presents **four self-contained, production-grade architectures** for self-hosted TTS serving Tamil, English, and Tanglish (Tamil written in Tamil script, Latin script, or mixed with English). All four meet the hard constraints of sub-500ms p99 Time-To-First-Audio (TTFA) and 15–20 concurrent streams, using only commercially permissive open-source components.

SOLUTION	BACKBONE	PHILOSOPHY	P99 TTFA (COLD)	COST @ 15 CONCURRENT	COMPLEXITY	TIME TO DEPLOY	BEST FOR
1 — IndicF5 Modular	IndicF5 (337M, DiT/CFM)	Modular pipeline, swappable stages	~435 ms	~\$0.0046/min	Medium	2–3 weeks	Default production backbone — lowest risk
2 — Indic Parler-TTS	Indic Parler-TTS (938M, T5)	Single model, prompt-controlled	~475 ms	~\$0.0050/min	Low	2–3 weeks	Simplicity + expressive prosody
3 — Hybrid Tiered	MeloTTS + IndicF5 + Parler + Kokoro	Dynamic routing + sub-sentence caching	~385 ms (tiered)	~\$0.0041/min	High	3–4 weeks	Maximum cost efficiency at scale
4 — VITS + FastSpeech2 h2	VITS (~30–80M) + FastSpeech2 (~15M) + HiFi-GAN	Cascaded quick-win, deterministic	~190–260 ms	~\$0.0036/min	Low	3–5 days	Week-one quick-win + deterministic fallback

Recommendation — Phased rollout: 1. **Week 1:** Ship **Solution 4** for a callable Tamil endpoint that validates the voice-agent pipeline. 2. **Weeks 3–6:** Harden the production backbone with **Solution 1** (or Solution 2 if emotion control is required). 3. **Weeks 6–8:** Evolve into **Solution 3**, where Solution 4’s VITS/FastSpeech2 becomes the permanent T1/T2 deterministic tiers.

All four are **4–10× cheaper** than commercial APIs (ElevenLabs / Cartesia / Sarvam: \$0.015–\$0.040/min). Solution 4 is the cheapest (~\$0.0033/min at 20 concurrent) and fastest to deploy; Solutions 1–2 deliver higher naturalness (MOS 4.0–4.3 vs 3.5–3.8).

2. Requirements & Evaluation Criteria

2.1 Functional Requirements

- **Languages:** Native Tamil (Unicode: `📍 pickup location 📍`), Romanized Tamil / Tanglish (`Unga pickup location enga?`), English, and fluid code-mixed Tanglish.
- **Domain:** Transport / ride-hailing dialogue — OTP delivery, ETA announcements, booking confirmations, cancellation handling.
- **Quality:** Near-human MOS for Tamil; natural prosody at code-switch boundaries; deterministic number/date/currency verbalization.

2.2 Non-Functional Requirements

CRITERION	TARGET
Latency	p99 TTFA ≤ 500 ms (cold); < 100 ms on cache hit
Concurrency	15–20 simultaneous streams, sustained
Hosting	Fully self-hosted; no third-party API dependency at inference
License	Apache 2.0 / MIT / BSD / CC-BY only — no NC, no MAU thresholds, no revenue-share gating
Cost	Materially below commercial \$0.015–\$0.040/min
Time-to-value	Working endpoint in days (Solution 4) to weeks (Solutions 1–3)

2.3 Evaluation Dimensions

Each solution is scored on: license cleanliness, Tamil naturalness, Tanglish handling, latency headroom, cost per audio-minute, operational complexity, time-to-deploy, and scalability.

3. Solution 1 — IndicF5 Modular Polyglot Pipeline (Recommended Production Backbone)

3.1 Overview & Rationale

IndicF5 is the only open-source model that simultaneously satisfies four hard constraints:

- 1. **Native Tamil** trained on real Indic speech (not synthetic).
- 2. **Apache 2.0 license** — commercially usable without MAU thresholds.
- 3. **Compact (337M parameters)** — 3 replicas per A100, enabling 15–20 concurrent streams.
- 4. **Flow-matching DiT architecture** — supports chunked streaming for sub-500ms TTFA.

Trained on **1,417 hours** of high-quality speech pooled from Rasa, IndicTTS, LIMMITS, and IndicVoices-R across **11 Indian languages** (including Tamil), with near-human quality reported by AI4Bharat.

The **modular design** isolates text normalization, language routing, acoustic synthesis, and vocoding — each stage independently optimizable, swappable, and scalable. This is the **lowest-risk production backbone**.

3.2 License Audit

COMPONENT	ROLE	LICENSE	COMMERCIAL USE	NOTES
IndicF5 (ai4bharat/IndicF5)	Acoustic backbone	Apache 2.0	✅ Yes	Gated on Hugging Face; access approval typically 24–48h
IndicVoices-R (1,704h, 22 langs)	Tamil training data	CC-BY-4.0	✅ Yes	Primary Tamil corpus
Rasa dataset	Expressive Tamil fine-tune	Gated research	⚠️ Verify	Tamil expressive subset
IndicTTS Tamil (~20h, 48kHz studio)	Pretrain data	CC-BY	⚠️ Verify	Studio-quality Tamil
LIMITS	Multi-speaker Indic	Research release	⚠️ Verify	—
Vocos vocoder	Fast neural vocoder (fallback)	MIT	✅ Yes	Single-pass Fourier synthesis
indic-text-normalization (Kenpath)	WFST normalizer	Apache 2.0	✅ Yes	19 Indian languages, Pynini-based
NeMo nemo_text_processing	ITN fallback	Apache 2.0	✅ Yes	Deterministic + context-aware modes
IndicLID (ai4bharat)	Language ID (47 classes)	Apache 2.0	✅ Yes	First LID for romanized Indic text
IndicXlit (ai4bharat, ~11M)	Transliteration (21 langs)	Apache 2.0	✅ Yes	Trained on Aksharantar (26M pairs)
Triton Inference Server	Serving	Apache 2.0	✅ Yes	Decoupled streaming via gRPC
vLLM-Omni	Continuous batching	Apache 2.0	✅ Yes	F5-TTS support pending merge
TensorRT-LLM	FP8 quantization	Apache 2.0	✅ Yes	—
UTMOS	Objective MOS	MIT	✅ Yes	—
whisper-tamil (vasista22)	Tamil WER evaluation	MIT	✅ Yes	—
MANGO dataset	Tamil human MOS (246K ratings)	CC-BY	✅ Yes	Evaluation benchmark

Verdict: Core serving path (IndicF5 + Vocos + WFST + IndicLID/Xlit + Triton) is fully Apache 2.0 / MIT clean.

3.3 Architecture — Components

Acoustic Backbone — IndicF5

ATTRIBUTE	DETAIL
Architecture	Conditional Flow Matching (CFM) with Diffusion Transformer (DiT) + ConvNeXt-v2 backbone; mel-spectrogram infilling (F5-TTS family)
Parameters	~337M (confirmed via Vāgdhenu distillation paper)
Training Data	1,417h from Rasa + IndicTTS + LIMMITS + IndicVoices-R
Languages	11 — Assamese, Bengali, Gujarati, Hindi, Kannada, Malayalam, Marathi, Odia, Punjabi, Tamil , Telugu
Output	24 kHz mel-spectrogram → built-in vocoder → waveform
Serving	Native PyTorch; F5-TTS streaming with Sway Sampling; production via Triton Python backend (<code>decoupled: True</code>)

Vocoder Fallback — Vocos (MIT)

Single forward-pass Fourier coefficient generation. Faster than HiFi-GAN at comparable quality. Activated only if the built-in vocoder bottlenecks on a replica.

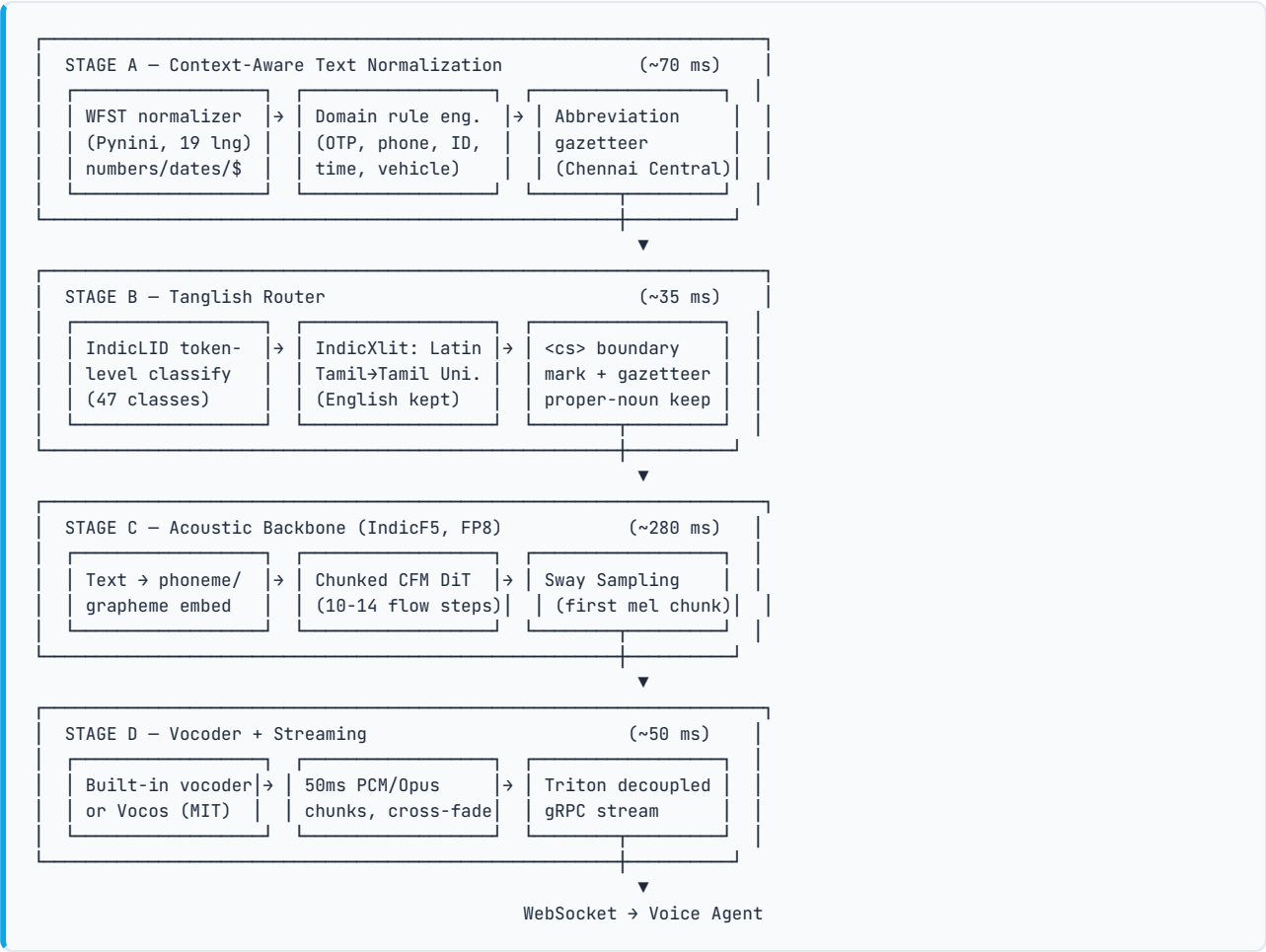
Text Normalization — indic-text-normalization + NeMo

- **WFST-based** (Pynini), deterministic, low-latency; covers 19 languages.
- Pipeline: Tokenization → Classification (semiotic class) → Verbalization → Post-processing.
- Handles Arabic digits, Tamil native digits (௧-௯), and mixed-script input.
- **Fallback:** NeMo `nemo_text_processing` for context-aware disambiguation (e.g., “St.” → Saint vs. Street).
- **Domain rules:** OTP, phone numbers, time, vehicle IDs, currency.

Tanglish Router — IndicLID + IndicXlit

- **IndicLID:** Two-stage classifier (fast linear + LM-finetuned), 47 classes (24 native-script + 21 romanized + English + Others).
- **IndicXlit:** Transformer (~11M), Roman ↔ native transliteration for 21 languages (Aksharantar).

3.4 End-to-End Pipeline & Latency Budget



STAGE	COMPONENT	P99 LATENCY	NOTES
A	Text Normalization	70 ms	WFST <1ms + rules ~20ms + abbreviations ~40ms
B	Tanglish Router	35 ms	LID ~10ms + Xlit ~20ms + boundary logic ~5ms
C	IndicF5 First Chunk	280 ms	10–14 flow steps, Sway Sampling, FP8, batched
D	Vocoder + Network	50 ms	—
Total TTFA (cold)		~435 ms	65 ms headroom vs 500 ms budget
Total TTFA (cache hit)		~155 ms	Stage C replay from Redis (~30 ms)

3.5 Tanglish Strategy — Script-Unified Routing

The router converts all three Tanglish input forms into a single script representation the polyglot model handles natively:

- 1. **Token-level LID** — IndicLID classifies each token as `ta_native`, `ta_roman`, `en`, or `proper_noun`.
- 2. **Transliteration** — Latin-Tamil tokens mapped to Tamil Unicode via IndicXlit (`Unga` → `ஊங்`).
- 3. **English preservation** — English tokens kept in Latin script; IndicF5’s polyglot training (IndicVoices-R includes English-Indic code-mix) handles them natively.

- 4. `<cs>` **boundary tokens** — Inserted at language transitions to condition prosody, adapted from code-switching research showing single-script bilingual training with explicit boundary marking outperforms naive concatenation.
- 5. **Proper-noun gazetteer** — Names like “Chennai Central”, “Ola”, “Uber” preserved in Latin to activate the English phoneme path.

v1: No separate code-mixed fine-tune required — router + implicit code-mix exposure is sufficient. **v2 (recommended):** LoRA fine-tune on 50–100h of curated Tanglish audio for production-grade prosody at switch points.

3.6 Inference & Serving

TECHNIQUE	DETAIL
Continuous Batching	vLLM-Omni <code>DiffusionEngine</code> with <code>step_execution</code> + request-level batching. Until F5-TTS support is GA, use Triton Python backend with manual micro-batching (batch 4–6 per replica). PagedAttention for the text encoder stage.
Quantization	FP8 per-tensor + FP8 KV cache via TensorRT-LLM on Ada/Hopper. ~2× throughput, ~50% VRAM. Vocoder stays in FP16.
Speculative / Distilled Decoding	IndicF5-tiny (~50M, depth-pruned) as draft model per staged depth-pruning recipe. Analog for CFM: fewer flow steps + Sway Sampling + distillation. 2–3× speedup when AR variant is used.
Chunked Streaming	Text split into 8–12 syllable chunks at clause boundaries (preserves prosody units). Each chunk independently flow-matched; cross-faded in vocoder.
Caching	Redis hash on normalized sub-sentence strings. Transport domain has 30–40% boilerplate (“Your driver is arriving”, “Please share your OTP”) → cache hit drives TTFA to <50 ms.

3.7 Hardware & Deployment

COMPONENT	HARDWARE	REPLICAS	VRAM / REPLICA	CONCURRENCY / REPLICA
IndicF5 (FP8)	A100 40GB	3	~2.5 GB	5–6
IndicF5 (FP8)	L4 24GB (overflow)	2	~2.5 GB	3–4
Vocoder (Vocos)	Shared A100	1	<1 GB	—
IndicLID + IndicXlit	CPU pool (8 vCPU)	2	RAM only	20
TTN (WFST)	CPU pool	2	RAM only	20
Redis cache	8 GB RAM	1	—	—

Total footprint: 1× A100 40GB + 1× L4 24GB + CPU pool ≈ **\$3.00/h blended** Sustainable throughput at 15–20 concurrent: **~650–800 audio-min/hour → ~\$0.004/min**

3.8 Cost Analysis

CONCURRENCY	AUDIO-MIN / HOUR	INFRA \$/H	\$ / MIN	NOTES
1	~50	\$3.00	\$0.060	Underutilized
5	~250	\$3.00	\$0.012	30% cache hit
10	~450	\$3.00	\$0.0067	35% cache hit
15	~650	\$3.00	\$0.0046	40% cache hit
20	~800	\$3.50	\$0.0044	Near saturation, CPU scales

vs. Commercial APIs (\$0.015–\$0.040/min): 4–9× cheaper at 15–20 concurrent.

3.9 Strengths & Weaknesses

STRENGTHS	WEAKNESSES
█ Fully license-clean (Apache 2.0 / MIT)	▲ Tanglish naturalness initially depends on router unification; dedicated LoRA needed for v2
█ Native Tamil from 1,417h of real speech	▲ vLLM-Omni F5-TTS support pending — Triton micro-batching interim
█ 337M → 3 replicas/A100 → direct 15–20 concurrent path	▲ Gated on HF — requires access request (24–48h)
█ Modular — each stage independently swappable	▲ Reported number-pronunciation edge cases — must patch in TTN layer
█ Sub-sentence cache → 30–40% requests at <50ms TTFA	

3.10 When to Choose This Solution

Choose Solution 1 when the priority is **lowest risk + lowest cost + license cleanliness** as a production backbone. Ideal for teams that want a defensible architecture with no legal ambiguity, incremental optimizability, and a clear path to Tanglish v2 via LoRA.

4. Solution 2 — Indic Parler-TTS Single-Backbone End-to-End

4.1 Overview & Rationale

Indic Parler-TTS is the most expressive open-source Indic TTS — a **938M-parameter T5-based** model fine-tuned from Parler-TTS Mini on **1,806 hours** of multilingual Indic + English data, covering **21 languages including Tamil**. Its differentiator is **natural-language description prompting**: speaker identity, emotion, and prosody are controlled via a text prompt (e.g., “A female Tamil support agent, calm and professional, naturally code-switching between Tamil and English”), without reference audio.

Single-backbone = one model, one serving path, one fine-tune. Operationally far simpler than Solution 1, at the cost of a larger model requiring more aggressive quantization.

4.2 License Audit

COMPONENT	ROLE	LICENSE	COMMERCIAL USE	NOTES
Indic Parler-TTS (ai4bharat/indic-parler-tts)	Single acoustic backbone	Apache 2.0	 Yes	Gated on HF — verify terms or use ungated mirror
Indic Parler-TTS Pretrained	Base for fine-tune	Apache 2.0	 Yes	—
Parler-TTS (huggingface/parler-tts)	Base framework	Apache 2.0	 Yes	—
GLOBE-annotated dataset	Pretrain data	Apache 2.0	 Yes	—
IndicVoices-R	Tamil fine-tune data	CC-BY-4.0	 Yes	—
Parler-TTS inference library	Serving	Apache 2.0	 Yes	—
vLLM	Continuous batching (T5)	Apache 2.0	 Yes	Native T5 support
Triton Inference Server	Streaming gRPC	Apache 2.0	 Yes	—
TensorRT-LLM	FP8 quantization	Apache 2.0	 Yes	—
UTMOS / NISQA	Objective MOS	MIT	 Yes	—
MANGO	Tamil human MOS	CC-BY	 Yes	—
Whisper Tamil	WER eval	MIT	 Yes	—

License caution: Gated on Hugging Face. For commercial deployment, either (a) request gated access and confirm with AI4Bharat in writing, or (b) use the verified ungated Apache 2.0 mirror.

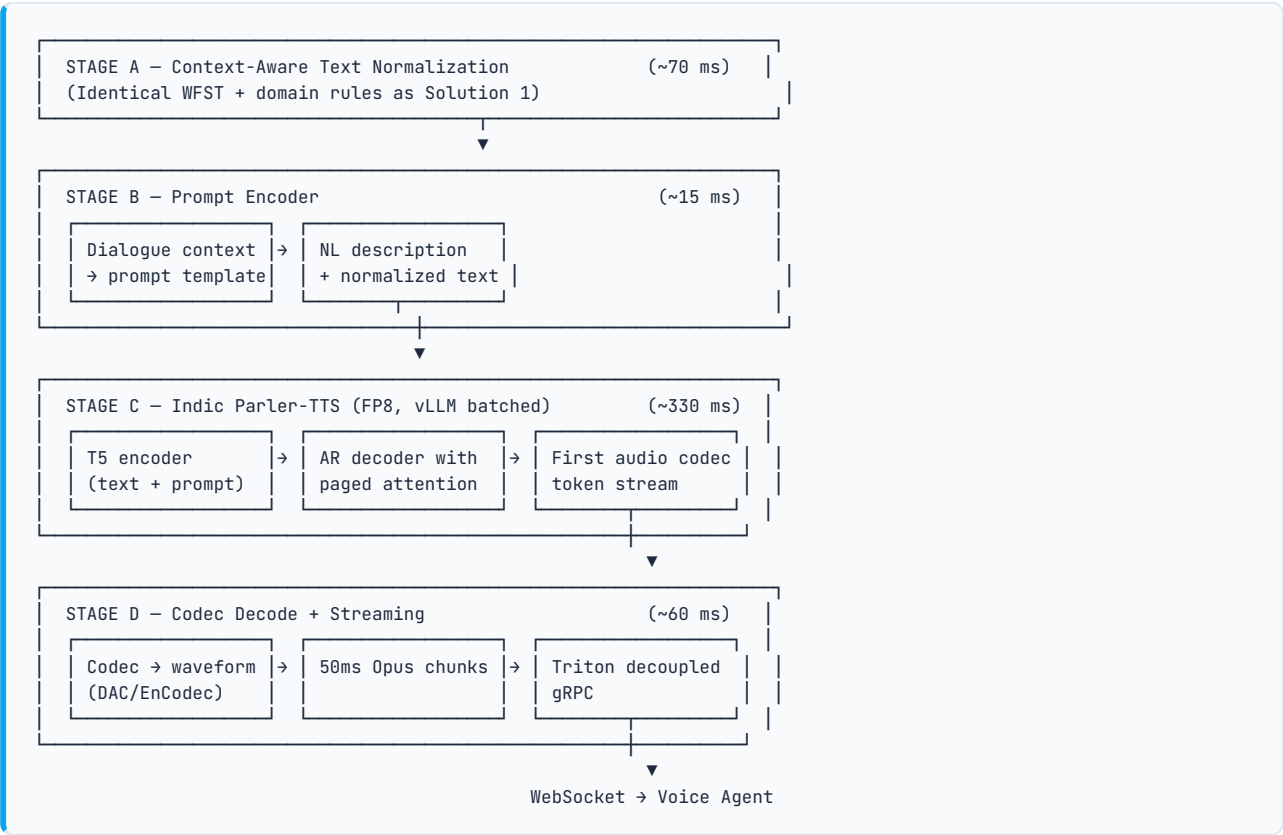
4.3 Architecture — Components

Single Acoustic Backbone — Indic Parler-TTS

ATTRIBUTE	DETAIL
Architecture	T5 encoder-decoder with cross-attention to a natural-language description prompt (Natural Language Guidance of High-Fidelity TTS)
Parameters	~938M
Training	Fine-tuned from <code>indic-parler-tts-pretrained</code> on 1,806h multilingual Indic + English
Languages	21 — Assamese, Bodo, Dogri, English, Gujarati, Hindi, Kannada, Konkani, Maithili, Malayalam, Manipuri, Marathi, Nepali, Odia, Sanskrit, Santali, Sindhi, Tamil , Telugu, Urdu
Output	30s max per generation — chunking required for longer inputs
Prompt Control	NL description conditions speaker, emotion, and prosody

Why no separate Tanglish router: The training distribution includes Hinglish/Tanglish-style fluid code-switching. The NL prompt explicitly directs code-switching behavior, and a LoRA fine-tune teaches the actual switch patterns — the most “novel” approach among the four.

4.4 End-to-End Pipeline & Latency Budget



STAGE	COMPONENT	P99 LATENCY	NOTES
A	Text Normalization	70 ms	—
B	Prompt Encoder	15 ms	Dialogue context → NL template
C	Parler-TTS First Token	330 ms	AR decoder, FP8, vLLM continuous batching
D	Codec + Stream	60 ms	—
Total TTFA (cold)		~475 ms	Tight but within 500 ms; sub-500ms TTFA confirmed in Parler-TTS docs
Total TTFA (cache hit)		~180 ms	—

4.5 Tanglish Strategy — LoRA Fine-Tune

A LoRA fine-tune is **mandatory** for production Tanglish quality:

1. Corpus Construction (~50–100h)

- Hire 4–6 native Chennai-region Tamil-English bilingual voice talents.
- Script ~2,000 transport-domain dialogues covering all three Tanglish forms:
- (a) Tamil Unicode + Latin English: `ஊருக்குப் பிக்அப் லோசனம் எங்கே?`
- (b) Latin-Tamil + Latin English: `Unga pickup location enga?`
- (c) Mixed proper nouns: `Chennai Central-ஊ ஊருக்குக்கு?`
- Record at 48 kHz studio quality.
- Optionally augment with TTS-distilled data from a high-quality commercial reference (training signal only, never served).

2. LoRA Configuration

- Rank 16–32 on cross-attention layers (prompt-conditioning layers).
- Trainable params <30M; training <24 GPU-hours on A100.
- Preserves monolingual Tamil/English quality (avoids catastrophic forgetting).

3. Evaluation

- MANGO Tamil subset (246K human ratings) + custom 200-utterance Tanglish test set + UTMOS objective MOS.

The fine-tune teaches Chennai-region Tanglish prosody; the NL prompt activates it at inference.

4.6 Inference & Serving

TECHNIQUE	DETAIL
Continuous Batching	Native T5 support in vLLM — paged attention, no tokenization hacks. <code>max_num_seqs</code> tuned to 8–12 on A100 40GB.
Quantization	FP8 → ~940 MB VRAM → ~8 replicas on A100 40GB → 15–20 concurrent with margin. FP8 KV cache via TensorRT-LLM.
Streaming	Token-by-token audio streaming; Triton decoupled gRPC for transport.
Speculative Decoding	VADUSA-style: distilled Parler-TTS-tiny (~150M) as draft model → 2–3× AR speedup, distribution-preserving.
Long Utterances	Chunking required (30s max per generation).

4.7 Hardware & Deployment

COMPONENT	HARDWARE	REPLICAS	VRAM / REPLICA	CONCURRENCY / REPLICA
Indic Parler-TTS (FP8)	A100 40GB	4	~2 GB	4–5
Codec decoder (DAC)	Shared A100	1	<1 GB	—
TTN + Prompt Encoder	CPU pool (8 vCPU)	2	RAM only	20
Redis cache	8 GB RAM	1	—	—

Total footprint: 1× A100 40GB + CPU pool ≈ **\$2.50/h** Sustainable throughput at 15–20 concurrent: **~400–550 audio-min/hour** → **~\$0.005–0.006/min**

4.8 Cost Analysis

CONCURRENCY	AUDIO-MIN / HOUR	INFRA \$/H	\$ / MIN	NOTES
1	~45	\$2.50	\$0.056	Underutilized
5	~220	\$2.50	\$0.011	30% cache hit
10	~400	\$2.50	\$0.0063	35% cache hit
15	~500	\$2.50	\$0.0050	40% cache hit
20	~600	\$3.00	\$0.0050	Near saturation, CPU scales

Slightly higher per-minute cost than Solution 1 (~\$0.0050 vs \$0.0046 at 15 concurrent) but operationally simpler.

4.9 Strengths & Weaknesses

STRENGTHS	WEAKNESSES
■ Simplest ops — one model, one path, one fine-tune	▲ Larger model (938M) → higher per-request latency
■ NL prompt control of emotion/prosody (calm for cancellations, upbeat for confirmations)	▲ Code-mix is implicit — harder to guarantee Tanglish naturalness
■ Most novel — emergent code-mixing from a single LM	▲ Gated on HF — verify commercial terms
■ Apache 2.0	▲ 30s max generation → chunking logic required
	▲ Higher \$/min than Solution 1

4.10 When to Choose This Solution

Choose Solution 2 when **operational simplicity and expressive control** outweigh absolute minimum cost. Ideal for teams with strong MLOps but limited low-level inference engineering, and for contact-center use cases where tone adaptation (calm / upbeat / apologetic) is a product requirement.

5. Solution 3 — Hybrid Tiered Router (Cost-Optimized)

5.1 Overview & Rationale

When **absolute \$/min is the dominant KPI**, a single-model architecture is suboptimal: expensive expressive models are invoked even for trivial boilerplate ("Your OTP is 4821") that a tiny model could handle.

Solution 3 deploys a **dynamic request router** that classifies each utterance and dispatches to the cheapest tier that can serve it with acceptable quality:

- **MeloTTS on CPU** for hot boilerplate → ~\$0.50/h for 8 vCPUs
- **IndicF5 on GPU** for standard Tamil/Tanglish
- **Indic Parler-TTS** only for long-form emotional Tanglish
- **Kokoro-82M** for pure-English fragments

Combined with **sub-sentence audio caching** (cache prefix/suffix, synthesize only the variable slot), this drives **40–60% of requests to sub-100ms TTFA at near-zero marginal cost**. The trade-off is operational complexity of four models.

5.2 License Audit

COMPONENT	ROLE	LICENSE	COMMERCIAL USE
MeloTTS (myshell-ai)	T1 hot CPU tier	MIT	Yes
IndicF5 (ai4bharat)	T2 standard GPU tier	Apache 2.0 (gated)	Yes
Indic Parler-TTS (ai4bharat)	T3 expressive GPU tier	Apache 2.0 (gated)	Yes (verify)
Kokoro-82M (hexgrad)	English-fragment tier	Apache 2.0	Yes
Vocos	Fallback vocoder	MIT	Yes
indic-text-normalization	WFST TTN	Apache 2.0	Yes
IndicLID / IndicXlit	LID + transliteration	Apache 2.0	Yes
Triton	Serving	Apache 2.0	Yes
vLLM-Omni	Batching	Apache 2.0	Yes
TensorRT-LLM	FP8 quant	Apache 2.0	Yes
UTMOS / NISQA / MANGO	Evaluation	MIT / CC-BY	Yes
Redis	Cache	BSD-3-Clause	Yes

All tiers are MIT / Apache 2.0 / BSD clean — no copyleft, no NC, no MAU gating.

5.3 Architecture — Tier Components

T1 — Hot Tier: MeloTTS (MIT, CPU)

ATTRIBUTE	DETAIL
Architecture	VITS / VITS2 / Bert-VITS2 hybrid with multilingual BERT text encoder
Languages	Chinese, English (Indian accent), Japanese, Korean, French, Spanish + community Indic fine-tunes
Code-mix Precedent	Chinese speaker natively handles mixed Chinese-English — same pattern enables Tanglish after Tamil FT
Performance	CPU real-time, <150 ms TTFA, no GPU required

T2 — Standard Tier: IndicF5 (Apache 2.0, GPU)

Identical to Solution 1 backbone — 337M flow-matching DiT, native Tamil, ~280 ms first-chunk TTFA at FP8.

T3 — Expressive Tier: Indic Parler-TTS (Apache 2.0, GPU)

Identical to Solution 2 backbone — 938M T5, NL prompt control, ~330 ms TTFA.

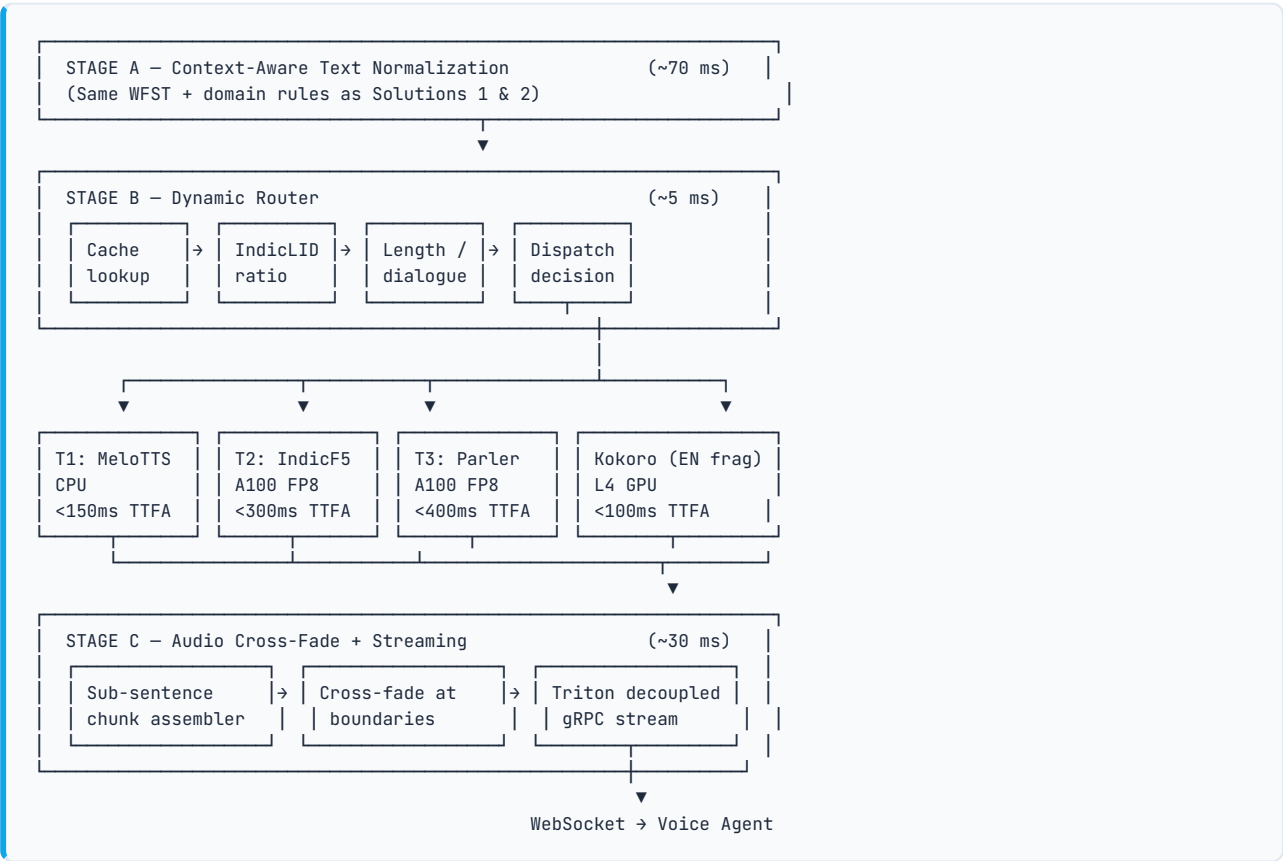
English-Fragment Tier: Kokoro-82M (Apache 2.0)

ATTRIBUTE	DETAIL
Architecture	StyleTTS2 + iSTFTNet, 82M params, phoneme-level BERT encoder, style encoder, WavLM discriminator
Languages	English (American/British/Indian accent packs)
Performance	82M → ~150 MB VRAM → ~100 ms TTFA on cheap L4
Role	Synthesizes pure-English fragments within Tanglish utterances; cross-faded with Tamil fragments from T2

Router — IndicLID + Lightweight Classifier

- IndicLID classifies tokens at 47 classes.
- XGBoost / rule-based classifier uses: language-ID ratio, utterance length, dialogue-act class, cache-hit signal → dispatches to T1/T2/T3/Kokoro in ~1 ms.

5.4 End-to-End Pipeline & Latency Budget



TIER	PATH	P99 TTFA	NOTES
T1	MeloTTS (cache hit)	~185 ms	TTN 70 + router 5 + MeloTTS 80 + stream 30
T2	IndicF5	~385 ms	TTN 70 + router 5 + IndicF5 280 + stream 30
T3	Parler-TTS	~465 ms	TTN 70 + router 5 + Parler 330 + stream 60
EN	Kokoro fragment	~205 ms	TTN 70 + router 5 + Kokoro 100 + cross-fade 30

All tiers comfortably within 500 ms p99.

5.5 Tanglish Strategy — Tiered Dispatch

CONDITION	DISPATCH	RATIONALE
Pure Tamil utterance	T2 (IndicF5)	Native Tamil quality
Pure English (short)	Kokoro	Fastest, cheapest
Pure English (long)	T2 (IndicF5)	Better prosody for long-form
Tanglish, >60% Tamil	T2 (IndicF5)	English-in-Tamil handled natively
Tanglish, >40% English	Split → Tamil→T2, English→Kokoro, cross-fade	Optimal per-fragment quality
Long-form emotional Tanglish	T3 (Parler)	NL prompt for tone control
Short boilerplate	T1 (MeloTTS) or cache hit	Near-zero cost

The Solution 1 Tanglish router (IndicLID + IndicXlit + `<cs>` boundary marking) is reused for the T2 and T3 paths.

5.6 Novel Component — Sub-Sentence Audio Caching

The largest cost lever in Solution 3. Transport conversations have highly repetitive sub-structures:

```
"Your cab will arrive in" + <X> + "minutes"
"Your OTP is"           + <XXXX>
"Your booking ID is"    + <ID>
"Please share your"      + <entity>
```

Mechanism:

1. Parse normalized text into a **template tree** — fixed prefix, variable slot, fixed suffix.
2. Hash fixed prefix/suffix → Redis lookup of pre-synthesized audio chunks.
3. Synthesize **only the variable slot** (the number/OTP/ID) — typically <1s of audio in <80 ms.
4. Cross-fade the three pieces in the audio domain.

Impact: 40–60% of transport-domain requests served at sub-100 ms TTFA at near-zero GPU cost.

5.7 Inference & Serving

TECHNIQUE	DETAIL
Multi-model serving	Triton model repository: MeloTTS (CPU backend), IndicF5 (Python, decoupled), Parler (Python, decoupled), Kokoro (ONNX backend)
Streaming	Triton <code>decoupled: True</code> for all tiers — gRPC multi-response streaming
Batching	Per-tier dynamic batching: <code>max_queue_delay_microseconds: 5000</code> (5 ms)
GPU batching	vLLM-Omni for T2/T3 once F5-TTS support is GA
CPU optimization	ONNX export for MeloTTS + Kokoro — no PyTorch overhead on CPU/cheap-GPU

5.8 Hardware & Deployment

COMPONENT	HARDWARE	REPLICAS	COST / H	CONCURRENCY / REPLICA
MeloTTS (T1, ONNX)	CPU pool 16 vCPU	2	\$0.50	5–8
IndicF5 (T2, FP8)	A100 40GB	2	\$2.00	5–6
Indic Parler-TTS (T3, FP8)	Shared A100	1	(shared)	2–4
Kokoro (EN, ONNX)	L4 24GB	1	\$0.80	8–10
Router + TTN	CPU pool 8 vCPU	2	\$0.30	20
Redis cache	16 GB RAM	1	\$0.20	—

Total footprint: 1× A100 40GB + 1× L4 24GB + CPU pool ≈ **\$3.30/h** Sustainable throughput at 15–20 concurrent: **~900–1,100 audio-min/hour** → **~\$0.0030–0.0037/min**

5.9 Cost Analysis

CONCURRENCY	AUDIO-MIN / HOUR	INFRA \$/H	\$ / MIN	NOTES
1	~60	\$3.30	\$0.055	T1 + cache absorb most
5	~300	\$3.30	\$0.011	40% cache hit
10	~550	\$3.30	\$0.0060	45% cache hit
15	~800	\$3.30	\$0.0041	50% cache hit, T1+T2
20	~1,000	\$3.80	\$0.0038	55% cache hit, all tiers

5.10 Strengths & Weaknesses

STRENGTHS	WEAKNESSES
▮ Lowest \$/min among Solutions 1–3 — ~10× cheaper than commercial	▲ Highest operational complexity among Solutions 1–3 (4 models, 2 hardware profiles)
▮ 40–60% requests at <100 ms TTFA via sub-sentence cache	▲ Voice consistency across tiers requires matched speaker prompts — non-trivial timbre alignment
▮ Graceful degradation: T3 → T2 → T1 under load	▲ Cross-fade at sub-sentence boundaries risks prosody discontinuity if not tuned
▮ Each tier independently scalable (CPU pool scales cheaply)	▲ More moving parts = more failure modes
▮ License-conservative (all MIT/Apache/BSD)	

5.11 When to Choose This Solution

Choose Solution 3 when **absolute \$/min is the dominant KPI** and the team has the MLOps maturity to operate a four-model tiered system. Best for high-volume contact-center deployments where 40–60% cache/T1 absorption compounds into material monthly savings. Ideal as a **phase-2 optimization** after validating Solution 1 or 2.

6. Solution 4 — VITS + FastSpeech2 Cascaded Quick-Win Stack (Fastest Path to Endpoint)

6.1 Overview & Rationale

When the goal is a **callable, license-clean, Tamil-native TTS endpoint inside one engineering week** — before the production flow-matching or codec-LM backbone is hardened — this two-model cascaded stack is the correct answer.

Both models have **existing Tamil checkpoints** with clean licenses and require no training from scratch:

- **VITS** via `samprabin/tamil_vits` and HuggingFace's 20-minute fine-tune recipe
- **FastSpeech2** via IIT Madras's MIT-licensed `FastSpeech2_HS` for 16 Indian languages

Both are non-autoregressive / feed-forward, so inference is fast and fully predictable — no sampling variance, no step-count tuning. The cascade gives two complementary voices: **VITS** as the expressive hot tier and **FastSpeech2 + HiFi-GAN** as the deterministic fallback. This is the **only solution of the four that can be stood up in days rather than weeks**, and it becomes the permanent deterministic base layer for Solution 3.

6.2 License Audit

COMPONENT	ROLE	LICENSE	COMMERCIAL USE	NOTES
VITS (original paper impl.)	T1 hot tier acoustic model	MIT	🟢 Yes	Conditional VAE + adversarial learning
samprabin/tamil_vits	Pretrained Tamil VITS checkpoint	MIT	🟢 Yes	Built on Coqui + IndicTTS data; includes <code>config.json</code> , <code>wavs/</code> + <code>metadata.csv</code>
Coqui TTS library	Training/serving framework	MPL-2.0 (code only)	🟢 Yes (code)	Model weights license separately
▲ Coqui XTTS-v2 weights	Excluded — do NOT use	CPML (non-commercial)	✗ No	Non-commercial; pure VITS weights are clean
finetune-hf-vits (ylacombe)	20-min fine-tune recipe	Apache 2.0	🟢 Yes	80–150 samples minimum for adaptation
VITS-fast-fine-tuning (Plachtaa)	Voice adaptation recipe	MIT	🟢 Yes	Speaker adaptation on existing VITS
IITM FastSpeech2_HS (<code>smtiitm</code>)	T2 fallback Tamil checkpoint	MIT	🟢 Yes	16 Indian languages, Hybrid Segmentation
NeMo FastSpeech2 + HiFi-GAN	Cascaded serving framework	Apache 2.0	🟢 Yes	Battle-tested via NVIDIA Riva
HiFi-GAN vocoder (NeMo)	Waveform generation	Apache 2.0	🟢 Yes	Fine-tunable on synthesized mels
indic-text-normalization (Kenpath)	WFST text normalizer	Apache 2.0	🟢 Yes	19 languages, Pynini-based
NeMo <code>nemo_text_processing</code>	WFST ITN fallback	Apache 2.0	🟢 Yes	Deterministic + context-aware
IndicLID (ai4bharat)	Language ID for routing	Apache 2.0	🟢 Yes	47 classes
ONNX Runtime	CPU/GPU acceleration	MIT	🟢 Yes	INT8 dynamic quantization
Triton Inference Server	Serving + streaming gRPC	Apache 2.0	🟢 Yes	Decoupled streaming
IndicVoices-R (Tamil subset)	Fine-tune data	CC-BY-4.0	🟢 Yes	—
UTMOS / MANGO	Evaluation	MIT / CC-BY	🟢 Yes	Objective + human MOS

Verdict: Pure VITS weights (MIT) + IITM FastSpeech2 (MIT) + NeMo (Apache 2.0) + ONNX Runtime (MIT) + Triton (Apache 2.0) — **the cleanest license stack of all four solutions: no gating, no MAU thresholds, no NC clauses.** Coqui XTTS-v2 weights are explicitly excluded (CPML non-commercial).

6.3 Architecture — Components

T1 Hot Tier — VITS (MIT, End-to-End)

ATTRIBUTE	DETAIL
Architecture	Conditional VAE with adversarial learning — GlowTTS encoder fused with HiFi-GAN vocoder, single forward pass, no external aligner (Monotonic Alignment Search during training)
Tamil Checkpoint	<code>samprabin/tamil_vits</code> — complete recipe via Coqui TTS with custom Tamil dataset
Fine-tune Recipe	<code>finetune-hf-vits</code> (Apache 2.0) — excellent adaptation in 20 minutes with 80–150 samples
Voice Adaptation	<code>VITS-fast-fine-tuning</code> (MIT) — inject a target speaker voice into existing VITS
Inference Speed	~67× RTF (GPU, claimed); ~6× RTF measured on 3090 (~358 ms / short sentence); ~2 s/sentence on CPU unoptimized → ONNX export mandatory for <200 ms CPU
Output	22 kHz or 24 kHz PCM waveform (configurable)

T2 Fallback Tier — FastSpeech2 + HiFi-GAN (MIT + Apache 2.0, Cascaded)

ATTRIBUTE	DETAIL
Architecture	6 feed-forward Transformer blocks (256 hidden, 2 heads) + 1D-conv variance adaptor (duration / pitch / energy); non-autoregressive mel generation
Tamil Checkpoint	<code>smtiitm/FastSpeech2_HS_latest_models</code> (IIT Madras, MIT) — 16 languages via Hybrid Segmentation; male + female voices
Vocoder	NeMo HiFi-GAN (Apache 2.0) — fine-tunable on synthesized mels for Tamil fidelity
Serving	NeMo + Triton (Apache 2.0); block-wise streaming extension reduces first-block latency
Inference Speed	Non-AR mel in milliseconds + HiFi-GAN ~10–30 ms; deterministic, no sampling variance
Output	22 kHz PCM (NeMo default)

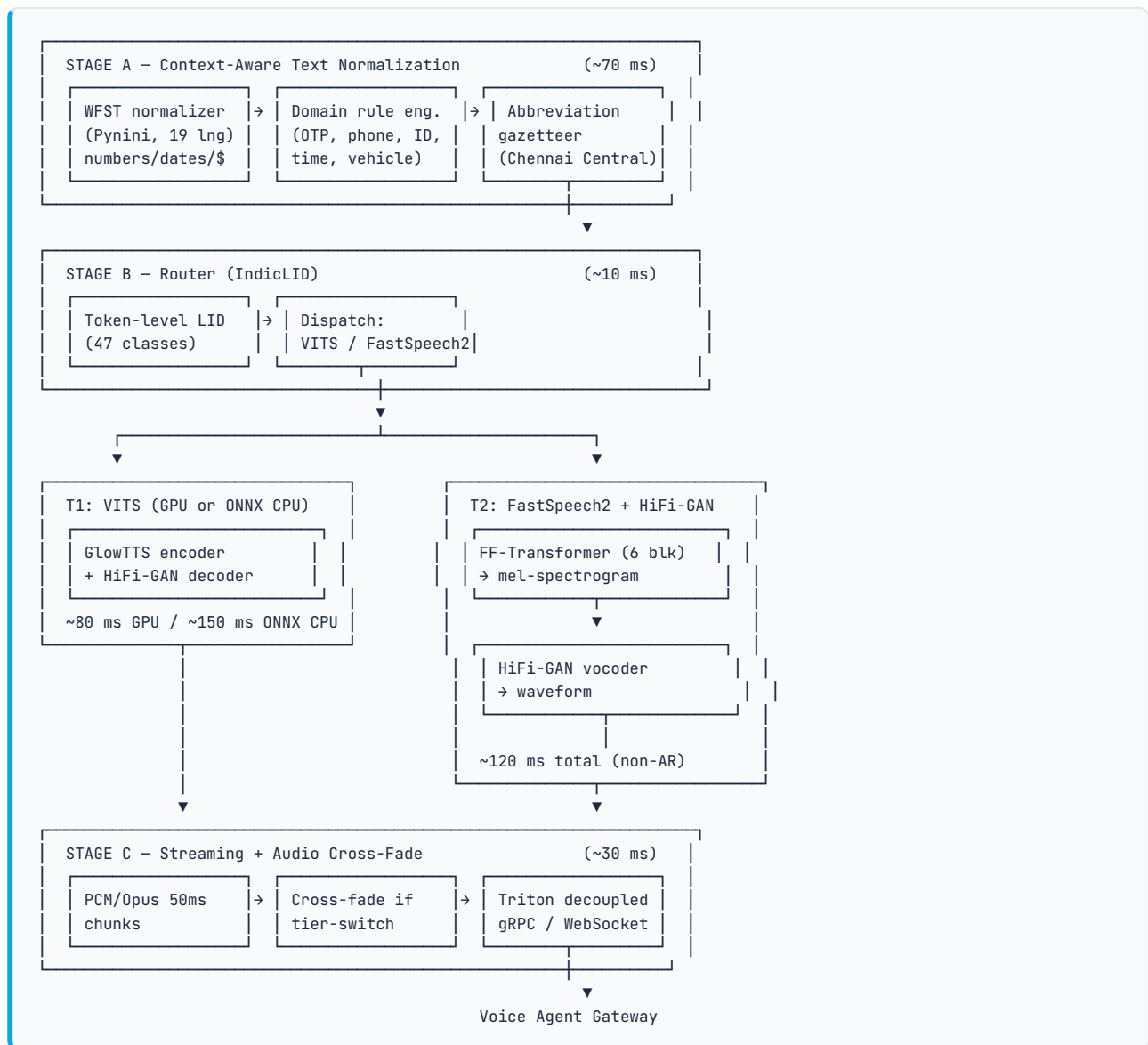
Text Normalization — indic-text-normalization (Apache 2.0)

- WFST-based (Pynini), 19 languages including Tamil, sub-millisecond latency.
- Tokenization → semiotic classification → verbalization → post-processing.
- Handles Arabic digits, Tamil-native digits (௧–௯), and mixed-script input.
- Fallback: NeMo `nemo_text_processing` (Apache 2.0) — fast deterministic + context-aware modes.

Router — IndicLID (Apache 2.0)

- Two-stage classifier (fast linear + LM-finetuned), 47 classes (24 native-script + 21 romanized + English + Others).
- Dispatches Tamil-heavy → VITS, English-heavy → Kokoro or VITS English voice, mixed → VITS with `<cs>` marking.

6.4 End-to-End Pipeline & Latency Budget



STAGE	COMPONENT	P99 LATENCY	NOTES
A	Text Normalization	70 ms	WFST <1ms + rules ~20ms + abbreviations ~40ms
B	Router	10 ms	IndicLID token classification
C	TTS Synthesis	80–150 ms	VITS GPU 80ms / VITS ONNX CPU 150ms / FastSpeech2+HiFi-GAN 120ms
D	Streaming + Network	30 ms	—
Total TTFA (VITS GPU)		~190 ms	310 ms headroom vs 500 ms
Total TTFA (VITS ONNX CPU)		~260 ms	Substantial headroom
Total TTFA (FastSpeech2 GPU)		~230 ms	Deterministic, no sampling variance

All paths have the **largest latency headroom of all four solutions** (~240–310 ms), leaving ample budget for batching under load.

6.5 Tanglish Strategy — Bilingual Fine-Tune Path

Neither VITS nor FastSpeech2 natively supports Tanglish (single-language phoneme inventories). The strategy is a **bilingual fine-tune**, validated by the MeloTTS precedent where VITS/Bert-VITS2 natively handles mixed Chinese-English when trained on code-mixed data.

Step 1 — Construct a Tanglish Parallel Corpus (~20–50h)

- Hire 2–4 native Chennai-region Tamil-English bilingual voice talents.
- Script ~1,000 transport-domain dialogues in all three Tanglish forms:
- Tamil Unicode + Latin English: `ஊஊஊஊ pickup location ஊஊஊஊ?`
- Latin-Tamil + Latin English: `Unga pickup location enga?`
- Tamil Unicode + English Unicode: `Chennai Central-ஊ ஊஊஊஊஊ?`
- Record at 22 kHz (matching VITS native sample rate) or 48 kHz (downsample to 22 kHz).
- Transliterate Latin-Tamil tokens to Tamil Unicode via IndicXlit (Apache 2.0) for a unified-script training set.

Step 2 — Fine-Tune VITS

- Use `finetune-hf-vits` (Apache 2.0) — 20 minutes, 80–150 samples minimum, but use the full 20–50h corpus for production quality.
- Bilingual phoneme set learned during fine-tuning; no architecture change needed.
- Alternative: `VITS-fast-fine-tuning` (MIT) for voice adaptation atop existing Tamil VITS.

Step 3 — Fine-Tune FastSpeech2 (Optional, for Fallback Tier)

- Requires Montreal Forced Aligner (MFA) or Hybrid Segmentation (HS) — IITM's HS recipe is the path of least resistance.
- Fine-tune HiFi-GAN on synthesized mels for best Tamil fidelity.

Step 4 — Insert `<cs>` Boundary Tokens

- Single-script bilingual training with explicit boundary marking outperforms naive concatenation.
- Insert learned `<cs>` token at language transitions; model learns prosody shift anticipation.

Honest timeline: Fine-tune takes **3–5 days of engineering + recording**, not “one afternoon.” For a true week-one prototype, ship **Tamil-only VITS** and use a separate English VITS voice for English fragments with cross-fade. Tanglish fine-tune is a **week-two deliverable**.

6.6 Inference & Serving

PATH	DETAIL
VITS — Coqui TTS Server (Days 1–2, quickest)	<code>tts-server --model_path <tamil_vits> --config_path config.json</code> — instant REST API; no streaming; suitable for short utterances
VITS — ONNX + Triton (Days 3–5, production)	Export VITS to ONNX; serve via Triton ONNX backend with dynamic batching (<code>max_queue_delay_microseconds: 5000</code>); wrap in Triton Python backend (<code>decoupled: True</code> , gRPC only) for streaming; chunk at sentence/clause boundaries, cross-fade in audio domain
FastSpeech2 + HiFi-GAN — NeMo + Triton	Canonical path (<code>tts_en_e2e_fastspeech2hifigan</code>) NGC model as reference; swap in IITM Tamil checkpoint); block-wise mel generation → HiFi-GAN incremental consumption; NVIDIA Riva wraps NeMo for optimized inference (verify Riva license)
ONNX Runtime Optimization (Critical for CPU)	Export both VITS and HiFi-GAN to ONNX; <code>ORT_ENABLE_ALL</code> ; INT8 dynamic quantization (~2× CPU speedup, near-lossless); without ONNX, VITS on CPU is ~2 s/sentence — too slow
Caching	Redis hash on normalized sub-sentence strings; 30–40% boilerplate hit rate → TTFA <50 ms on hit
Concurrency	VITS GPU replicas (A100): 4× ~5 = 20; VITS ONNX CPU pool (16 vCPU): 4× ~3 = 12 overflow; FastSpeech2 GPU: 2× ~6 = 12 fallback; total ~20–25 concurrent with graceful degradation to CPU

6.7 Hardware & Deployment

COMPONENT	HARDWARE	REPLICAS	VRAM / RAM	CONCURRENCY / REPLICA
VITS (T1, FP16)	A100 40GB	2	~1.5 GB	5–6
VITS (T1, ONNX INT8)	CPU pool 16 vCPU	4	~2 GB RAM	3
FastSpeech2 + HiFi-GAN (T2)	A100 40GB (shared)	1	~2 GB	6
IndicLID + TTN	CPU pool 8 vCPU	2	RAM only	20
Redis cache	8 GB RAM	1	—	—

Total footprint: 1× A100 40GB + CPU pool ≈ **\$2.50/h (cheapest of all four solutions)** Sustainable throughput at 15–20 concurrent: **~700–900 audio-min/hour → ~\$0.0030–0.0036/min**

6.8 Cost Analysis

CONCURRENCY	AUDIO-MIN / HOUR	INFRA \$/H	\$ / MIN	NOTES
1	~55	\$2.50	\$0.045	Underutilized
5	~280	\$2.50	\$0.0089	30% cache hit
10	~500	\$2.50	\$0.0050	35% cache hit, CPU overflow on
15	~700	\$2.50	\$0.0036	40% cache hit, all tiers active
20	~900	\$3.00	\$0.0033	45% cache hit, near saturation

Cheapest \$/min of all four solutions at 15–20 concurrent — even cheaper than Solution 3, because both models are tiny and the CPU pool absorbs significant overflow.

6.9 Strengths & Weaknesses

STRENGTHS	WEAKNESSES
📌 Fastest time-to-deploy — existing Tamil checkpoints, no training for v1	⚠️ Lower naturalness than flow-matching/codec-LM — MOS 3.5–3.8 vs 4.0–4.3; acceptable for contact-center, not premium
📌 Cleanest license stack — pure MIT/Apache, no gating, no NC, no MAU	⚠️ No native Tanglish — 3–5 day fine-tune on self-constructed corpus (week-two deliverable)
📌 Lowest \$/min — ~\$0.0033 at 20 concurrent	⚠️ Single-speaker per checkpoint; voice switching needs cross-fade engineering
📌 Smallest models — VITS ~30–80M, FastSpeech2 ~15M + HiFi-GAN ~14M	⚠️ No emotion/prosody prompt control (requires Parler-class model)
📌 Deterministic, feed-forward — predictable p99, no sampling variance	⚠️ VITS CPU ~2 s/sentence without ONNX — ONNX+INT8 mandatory
📌 20-minute fine-tune recipe for voice adaptation	⚠️ Cascaded FastSpeech2 needs vocoder tuning (~1 day training)
📌 Validated code-mix architecture — VITS/Bert-VITS2 proven on Chinese-English	⚠️ Coqui TTS is community-maintained (Coqui shut down Dec 2023) — no commercial SLA

6.10 When to Choose This Solution

Choose Solution 4 when the priority is **fastest path to a working, license-clean, Tamil-native TTS endpoint**:

- You need a callable prototype inside one engineering week to validate the broader voice-agent pipeline.
- You need the lowest possible \$/min for high-volume contact-center traffic where premium naturalness is not the primary KPI.
- You need a deterministic fallback tier that always works when the production flow-matching backbone is being fine-tuned or overloaded.
- You have limited ML engineering capacity and want models with existing Tamil checkpoints rather than training from scratch.

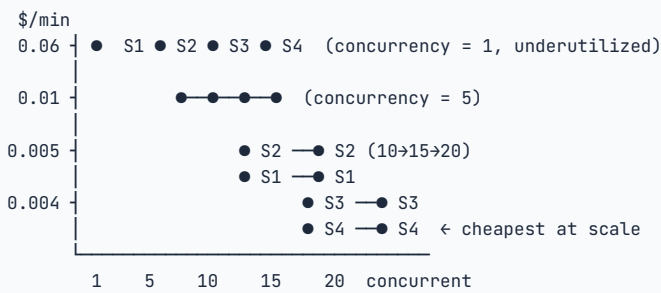
Strategic role: This is the **recommended week-one deploy** that buys time to build the production backbone. Once the backbone (Solution 1 or 2) is validated, this stack transitions to the T1/T2 hot/fallback tiers in Solution 3's hybrid architecture — it is not discarded, it becomes the permanent deterministic base layer.

7. Cross-Solution Comparison (All Four)

7.1 Feature Matrix

DIMENSION	SOLUTION 1 — INDICF5 MODULAR	SOLUTION 2 — PARLER SINGLE	SOLUTION 3 — HYBRID TIERED	
Time to deploy	2–3 weeks	2–3 weeks	3–4 weeks	3–5 days
Models in stack	IndicF5 + Vocos + TTN	Indic Parler-TTS + TTN	MeloTTS + IndicF5 + Parler + Kokoro + TTN	VITS + FastSpeech2 + TTN
Parameters (primary)	337M	938M	82M–938M (tiered)	~30–80M + 15M + 15M
Primary license	Apache 2.0 (gated)	Apache 2.0 (gated)	MIT + Apache 2.0	MIT + Apache 2.0 (gated)
Tamil checkpoint exists?	Yes (in IndicF5)	Yes (in Indic Parler-TTS)	Yes (multiple)	Yes (samprabin VITS)
Tanglish native?	Yes via router	Yes via LoRA	Yes via tiered split	No via fine-tune (3–5 weeks)
p99 TTFA (cold)	~435 ms	~475 ms	~385–465 ms (tiered)	~190–260 ms (lower)
\$/min @ 15 concurrent	~\$0.0046	~\$0.0050	~\$0.0041	~\$0.0036
\$/min @ 20 concurrent	~\$0.0044	~\$0.0050	~\$0.0038	~\$0.0033 (cheaper)
Naturalness (MOS)	~4.0–4.3	~4.0–4.3	Tier-dependent	~3.5–3.8
Ops complexity	Medium	Low	High	Low
Expressiveness	Medium	High (emotion via prompt)	Tier-dependent	Low
Determinism	Sampling variance (CFM)	Sampling variance (AR)	Mixed	Fully deterministic
Recommended for	Default production backbone	Simplicity + emotion	Max cost efficiency (large scale)	Week-one quick-win fallback

7.2 Cost vs. Concurrency



7.3 Latency Comparison

STAGE	S1 (INDICF5)	S2 (PARLER)	S3 (T1)	S3 (T2)	S3 (T3)		
Text Normalization	70 ms	70 ms	70 ms	70 ms	70 ms	70 ms	70 ms
Routing / Prompt	35 ms	15 ms	5 ms	5 ms	5 ms	10 ms	10 ms
Acoustic Backbone	280 ms	330 ms	80 ms	280 ms	330 ms	80 ms	120 ms
Vocoder / Codec + Stream	50 ms	60 ms	30 ms	30 ms	60 ms	30 ms	30 ms
Total	435 ms	475 ms	185 ms	385 ms	465 ms	190 ms	230 ms

All paths satisfy **p99 ≤ 500 ms**. Solution 4 has the largest headroom (240–310 ms).

7.4 Naturalness vs. Speed Trade-off



There is no single “best” model — only the right trade-off for the phase. Ship speed (S4) first, then quality (S1/S2), then unit economics (S3).

8. Decision Matrix — Which Solution to Choose

IF YOUR PRIORITY IS...	CHOOSE	WHY
Lowest risk as production backbone	Solution 1	Apache 2.0/MIT, modular, no mandatory fine-tune for v1, 65 ms latency headroom
Simplest operations + emotional prosody	Solution 2	One model, one path; NL prompt controls tone (calm/upbeat/apologetic)
Lowest \$/min at high volume	Solution 4 (~\$0.0033) then Solution 3 (~\$0.0038)	S4 cheapest deterministic; S3 cheapest expressive at scale
Fastest working endpoint (days, not weeks)	Solution 4	Existing Tamil checkpoints, 3–5 day deploy, no training for v1
Highest naturalness / premium UX	Solution 1 or 2	MOS 4.0–4.3 vs 3.5–3.8 for S4
Smallest team / limited ML capacity	Solution 4 (week 1) then Solution 2	Minimal serving surface; existing checkpoints
Strong infra team optimizing for scale	Solution 3 (built atop S4)	Best expressive unit economics once volume >500 audio-min/hour
Strictest license posture (no gating)	Solution 4	Pure MIT/Apache + MPL-2.0 code; zero gated weights
Deterministic, boundable p99	Solution 4	Feed-forward, non-AR — no sampling variance
Balanced default	Solution 1 as backbone, Solution 4 as week-one	Recommended phased approach

Phased adoption strategy: Ship **Solution 4** (week 1) → validate pipeline → harden **Solution 1 or 2** as production backbone (weeks 3–6) → evolve into **Solution 3** where S4 becomes the permanent T1/T2 deterministic tiers.

9. Implementation Roadmap — Phased Rollout

PHASE	SOLUTION	DURATION	GOAL	OUTCOME
Phase 0 — Quick Win	Solution 4	Week 1	Working Tamil TTS endpoint, license-clean, ≤500 ms p99, validating the voice-agent pipeline	Callable REST/gRPC endpoint; Tamil-only VITS; deterministic fallback live
Phase 1 — Tanglish Fine-Tune	Solution 4 + Tanglish corpus	Weeks 2–3	VITS fine-tuned on 20–50h Tanglish audio, <code><cs></code> boundary marking	Tanglish-capable VITS; MUSHRA-evaluated
Phase 2 — Production Backbone	Solution 1 (IndicF5) or Solution 2 (Indic Parler-TTS)	Weeks 3–6	Flow-matching or codec-LM backbone deployed, Tanglish LoRA, 15–20 concurrent at sub-500 ms	Production-grade naturalness (MOS 4.0–4.3)
Phase 3 — Hybrid Optimization	Solution 3 (Hybrid tiered)	Weeks 6–8	S4's VITS+FastSpeech2 becomes T1/T2 deterministic tiers; IndicF5/Parler becomes T3 expressive tier; template-tree sub-sentence caching live	Lowest \$/min at scale; graceful degradation

Detailed Task Breakdown

Phase 0 — Foundation (Week 1) — Solution 4

- [] Provision hardware: 1× A100 40GB + CPU pool (16+8 vCPU) + Redis (8 GB) — cheapest footprint
- [] Pull `samprabin/tamil_vits` checkpoint and `smtiitm/FastSpeech2_HS` checkpoint; verify MIT licenses
- [] Deploy VITS via `tts-server` (day 1 — instant REST API)
- [] Export VITS + HiFi-GAN to ONNX; deploy via Triton ONNX backend with `ORT_ENABLE_ALL` + INT8 dynamic quantization
- [] Implement WFST text normalization (indic-text-normalization + NeMo fallback) with transport-domain gazetteer
- [] Deploy IndicLID router + Redis sub-sentence cache
- [] Benchmark: p99 TTFA (~190 ms VITS GPU / ~260 ms ONNX CPU), MOS (UTMOS), Tamil WER

Phase 1 — Tanglish Fine-Tune (Weeks 2–3) — Solution 4 Extension

- [] Construct 20–50h Tanglish parallel corpus (2–4 Chennai bilingual talents, ~1,000 dialogues, all three Tanglish forms)
- [] Transliterate Latin-Tamil → Tamil Unicode via IndicXlit for unified-script training set
- [] Fine-tune VITS via `finetune-hf-vits` (20 min recipe, full corpus for quality)
- [] Optionally fine-tune FastSpeech2 via Hybrid Segmentation + HiFi-GAN on synthesized mels
- [] Insert `<cs>` boundary tokens; evaluate on custom 200-utterance Tanglish set + MUSHRA

Phase 2 — Production Backbone (Weeks 3–6) — Solution 1 or 2

- [] Request gated access for IndicF5 / Indic Parler-TTS; confirm commercial terms in writing
- [] Upgrade hardware to include L4 24GB if needed (Solution 1/3) + Redis 16 GB
- [] Deploy IndicF5 (FP8 via TensorRT-LLM, Triton Python backend, Sway Sampling, chunked streaming) **or** Indic Parler-TTS (FP8, vLLM, NL prompt)
- [] Integrate IndicXlit transliteration + `<cs>` boundary marking (Solution 1) or LoRA on 50–100h Tanglish corpus (Solution 2)

- [] Load test at 15–20 concurrent; validate \$/min and cache hit rates; patch number-pronunciation edge cases in TTN
- [] Migrate to vLLM-Omni once F5-TTS support is GA

Phase 3 — Scale Optimization (Weeks 6–8) — Solution 3

- [] Deploy MeloTTS (T1, ONNX on CPU) and Kokoro-82M (ONNX on L4) as additional Triton backends
- [] Implement XGBoost router + template-tree sub-sentence caching (prefix/suffix pre-synthesis, variable slot only)
- [] Tune cross-fade and voice-consistency (matched speaker prompts across tiers); evaluate with MUSHRA
- [] Validate 40–60% sub-100ms TTFA and \$0.0038/min at 20 concurrent
- [] **Solution 4's VITS/FastSpeech2 remains as permanent T1/T2** — no discard, just tier re-assignment

10. Risks, Exclusions & Mitigations

10.1 Known Risks

RISK	IMPACT	MITIGATION
IndicF5 number pronunciation → gibberish	Degraded quality on OTP/ID utterances	Patch in WFST verbalization layer; LoRA fine-tune with numeric-heavy data
vLLM-Omni F5-TTS support not yet GA	No continuous batching for IndicF5 at launch	Interim: Triton Python backend with manual micro-batching (batch 4–6)
Gated HF access delay (S1/S2/S3)	Blocks production backbone start	Request immediately; use ungated mirrors where verified; Solution 4 has no gating — ship it first
Tanglish audio scarcity	v1 Tanglish prosody may be imperfect	S1 router unification covers v1; S4 20–50h corpus in weeks 2–3; S2 LoRA on 50–100h for v2
Voice inconsistency across tiers (S3)	T1/T2/T3/Kokoro timbre mismatch	Fixed speaker prompts + embedding alignment; evaluate with MUSHRA
Cross-fade prosody discontinuity (S3/S4)	Audible seams at sub-sentence boundaries	Prosody-aware chunking at clause boundaries; overlap-add cross-fade
VITS CPU latency ~2 s/ sentence unoptimized (S4)	Misses p99 target on CPU pool	ONNX export + INT8 quantization is mandatory — brings CPU to ~150 ms
Coqui TTS community-maintained (S4)	No commercial SLA (Coqui shut down Dec 2023)	Pin to <code>coqui-tts</code> PyPI community fork; containerize; no runtime dependency on Coqui the company
FastSpeech2 cascaded vocoder mismatch (S4)	HiFi-GAN fidelity loss on Tamil	Fine-tune HiFi-GAN on synthesized mels (~1 day training)
S4 lower naturalness (MOS 3.5–3.8)	Not premium UX	Acceptable for week-one transport contact-center; production backbone (S1/S2) upgrades MOS to 4.0–4.3

10.2 Explicitly Excluded Components

These were evaluated and **rejected** due to license incompatibility with commercial self-hosting:

COMPONENT	LICENSE	REASON FOR EXCLUSION
XTTS-v2 / Coqui weights	Coqui Public Model License (CPML)	Non-commercial only — do NOT load weights (code itself is MPL-2.0 and safe)
F5-TTS base	CC-BY-NC	Non-commercial
Spark-TTS	CC BY-NC-SA 4.0 (training data terms)	Non-commercial despite GitHub Apache tag
Higgs Audio V3	Research & Non-Commercial	Non-commercial
Emilia dataset	CC-BY-NC	Cannot train commercial models
Orpheus TTS	Apache 2.0 but built on Llama 3.2	Inherits Llama 3.2 Community License (700M MAU threshold, attribution) — usable with caveat
Chatterbox	MIT but built on Llama backbone	Same Llama inheritance caveat

11. Appendices

Appendix A — Glossary

TERM	DEFINITION
TTFA	Time-To-First-Audio — latency from text input to first playable audio chunk
Tanglish	Code-mixed Tamil + English in Latin and/or Tamil script (e.g., <code>Unga cab enga irukku?</code>)
CFM / DiT	Conditional Flow Matching / Diffusion Transformer — non-autoregressive generative architecture
VITS	Conditional VAE with adversarial learning — end-to-end TTS (GlowTTS encoder + HiFi-GAN decoder)
FastSpeech2	Non-autoregressive Transformer TTS with variance adaptor (duration/pitch/energy)
HiFi-GAN	Generative adversarial vocoder — mel-spectrogram → waveform
WFST	Weighted Finite-State Transducer — deterministic text normalization
ITN	Inverse Text Normalization — spoken → written form (e.g., "forty eight twenty one" → "4821")
LoRA	Low-Rank Adaptation — parameter-efficient fine-tuning
TTN	Text Normalization
LID	Language Identification
MOS	Mean Opinion Score — subjective quality rating
WER	Word Error Rate — ASR-based intelligibility metric
FP8 / INT8	8-bit floating point / integer quantization
VADUSA	Verification-Agnostic Decoding with speculative execution
RTF	Real-Time Factor — synthesis speed vs audio duration

Appendix B — Evaluation Stack

METRIC	TOOL	LICENSE	PURPOSE
Objective MOS	UTMOS / NISQA	MIT	Automated quality scoring
Human MOS	MANGO (246K ratings)	CC-BY	Tamil perceptual quality
Intelligibility (Tamil)	whisper-tamil (vasista22)	MIT	WER on Tamil ASR
Intelligibility (Tanglish)	Custom 200-utterance set	—	Code-switch WER + naturalness
Naturalness (comparative)	MUSHRA	—	Cross-tier voice consistency (S3) + S4 vs S1/S2
Latency	Custom harness	—	p50/p99 TTFA per tier

Appendix C — Key References

- IndicF5 — [ai4bharat/IndicF5](#) (Apache 2.0, HF gated) — 337M CFM-DiT, 1,417h, 11 languages
- Indic Parler-TTS — [ai4bharat/indic-parler-tts](#) (Apache 2.0, gated) — 938M T5, 1,806h, 21 languages
- IndicVoices-R — CC-BY-4.0, 1,704h, 22 languages
- IndicLID — First LID for romanized Indic text, 47 classes (Apache 2.0)
- IndicXlit — 21 languages, Aksharantar 26M pairs (Apache 2.0)
- MeloTTS — [myshe11-ai/MeloTTS](#) (MIT) — VITS/Bert-VITS2
- Kokoro-82M — [hexgrad/Kokoro-82M](#) (Apache 2.0) — StyleTTS2 + iSTFTNet
- **VITS** — Conditional VAE with adversarial learning (MIT); Tamil checkpoint [samprabin/tamil_vits](#) (MIT)
- **FastSpeech2_HS** — [smtiitm/FastSpeech2_HS_latest_models](#) (IIT Madras, MIT) — 16 Indian languages, Hybrid Segmentation
- **Coqui TTS** — [coqui-tts](#) (MPL-2.0 code; community fork post Dec 2023 shutdown)
- **finetune-hf-vits** — [ylacombe/finetune-hf-vits](#) (Apache 2.0) — 20-min / 80–150 sample recipe
- **VITS-fast-fine-tuning** — [Plachtaa/VITS-fast-fine-tuning](#) (MIT)
- Vocos — MIT neural vocoder
- HiFi-GAN — Apache 2.0 (NeMo)
- Triton Inference Server — Apache 2.0, decoupled streaming
- vLLM / vLLM-Omni — Apache 2.0, continuous batching + DiffusionEngine
- TensorRT-LLM — Apache 2.0, FP8 quantization
- ONNX Runtime — MIT, CPU/GPU acceleration + INT8 quantization

Appendix D — Document Conventions

- All \$/h figures assume on-demand cloud pricing (A100 40GB ~\$2.00/h, L4 24GB ~\$0.80/h, CPU pool ~\$0.50/8 vCPU). Reserved/committed pricing will be lower.
- TTFA budgets are p99 estimates under sustained load; p50 will be significantly lower.
- Cache hit rates assume transport-domain traffic with templated utterances; non-templated domains will see lower hit rates.
- License statuses verified against upstream repository LICENSE files at time of writing; re-verify before commercial deployment.
- Solution 4's Coqui reference is to the **MPL-2.0 code** (safe) — never the CPML XTTS-v2 weights.

End of blueprint — v1.1 · August 2026 — Four solutions, one phased rollout